Proof of Human Intent: Cryptographically Verifiable Human Approval for AI-Driven Development

Ikko Eltociear Ashimine
Independent Researcher
ORCID: 0000-0002-3576-6677

***Abstract*—With the rise of autonomous AI agents capable of proposing and executing software changes, traditional human-in-the-loop assumptions no longer hold. As these agents increasingly automate code generation, review, and deployment, the provenance of "human intent"—the accountability behind critical decisions—is becoming obscured. Current systems lack mechanisms to cryptographically verify that a human—rather than an automated process—authorized specific actions such as merging pull requests or deploying to production. We present Proof of Human Intent (PoHI), a protocol that combines zero-knowledge proofs of personhood (e.g., World ID), decentralized identifiers (DIDs), verifiable credentials (VCs), and transparency logs (SCITT) to create tamper-evident, machine-verifiable records of human approval. Our architecture addresses three fundamental questions: (1) *who* approved an action (unique human verification), (2) *what* was approved (cryptographic binding to specific commits), and (3) *when* it was approved (immutable timestamping). We implement a proof-of-concept integration with GitHub Actions and demonstrate that PoHI prevents unauthorized automated merges with negligible latency overhead (<2 seconds total machine time). The reference implementation is available at https://github.com/pohi-protocol/pohi.**

***Index Terms*—AI agents, human-in-the-loop, proof of personhood, software supply chain security, zero-knowledge proofs, decentralized identity, verifiable credentials, transparency logs**

I. Introduction

The rapid advancement of AI-driven development tools has fundamentally transformed software development workflows. Tools such as GitHub Copilot, Claude Code, and autonomous coding agents can now generate, review, and even propose merging code changes with minimal human intervention. While this automation dramatically increases developer productivity, it raises critical questions about accountability: *Who approved this code? Was it a human or an AI? Can we prove it?*

Consider a scenario where an AI agent autonomously creates a pull request, reviews it using another AI system, and merges it into production—all without meaningful human oversight. In 2024, GitHub Copilot assists in over 46% of code written by developers using the tool. By 2025, autonomous coding agents like Devin, Claude Code Agent, and GPT Engineer can complete entire development tasks end-to-end. This trend points toward a future where AI systems not only write code but also manage significant portions of the software lifecycle.

The accountability gap becomes critical when we consider:

- **Security incidents**: If malicious code is introduced, who is responsible?
- **Regulatory compliance**: Industries like finance and healthcare require human approval for critical changes.
- **Audit requirements**: SOC 2, ISO 27001, and similar frameworks mandate traceable approval processes.
- **Legal liability**: Contracts may require human sign-off for software releases.

Current solutions rely on social conventions (e.g., requiring reviews) or process controls (e.g., protected branches), but these are easily circumvented and lack cryptographic verifiability. There is no mechanism to prove, after the fact, that a genuine human—not a bot or automated system—approved a specific action.

A. Contributions

Our contributions are as follows:

- We identify the "human approval verification gap" in AI-driven development and formalize the threat model.
- We propose the PoHI architecture that integrates proof-of-personhood, decentralized identity, and transparency logging.
- We implement a proof-of-concept GitHub Action that enforces human approval verification.
- We analyze the security properties of PoHI against relevant attack vectors.

II. Background and Related Work

A. AI Agents in Software Development

The evolution of AI in software development can be characterized in three phases:

Phase 1 (2021-2023): Completion-based assistants. GitHub Copilot and similar tools provide inline code suggestions. The human developer remains in control of all actions.

Phase 2 (2024-2025): Agentic assistants. Tools like Claude Code, Cursor, and Windsurf can execute multi-step tasks, run commands, and modify files autonomously within developer-defined boundaries.

Phase 3 (2025+): Autonomous agents. Systems like Devin and OpenAI's Operator can complete entire tasks independently, including creating pull requests and interacting with external services.

This progression shifts developers from "implementers" to "approvers," creating an urgent need for verifiable human oversight.

B. Proof of Personhood

Proof of Personhood (PoP) systems aim to verify that an entity is a unique human without revealing their identity. While PoHI is designed to be provider-agnostic, we use World ID as a concrete instantiation in this work due to its strong cryptographic guarantees. Notable approaches include:

World ID [1] uses iris biometrics via specialized hardware (Orb) to create a unique identifier. Zero-knowledge proofs allow users to prove their humanity without revealing biometric data. World ID provides both "Device" level (phone-based) and "Orb" level (biometric) verification.

BrightID uses a social graph approach where existing verified humans vouch for new members. While more accessible, it is more susceptible to Sybil attacks through collusion.

Gitcoin Passport aggregates multiple identity signals (social accounts, on-chain activity) into a "humanity score." This provides gradual verification but lacks the binary uniqueness guarantee of biometric systems.

We choose World ID as our primary PoP provider due to its strong Sybil resistance and privacy-preserving properties through zero-knowledge proofs.

C. Software Supply Chain Security

Recent initiatives address software supply chain integrity:

SCITT (Supply Chain Integrity, Transparency, and Trust) [5] is an IETF draft architecture for maintaining append-only transparency logs of supply chain claims. It provides a framework for recording and verifying attestations about software artifacts.

Sigstore [6] enables keyless code signing using OIDC identity providers. It provides transparency logs (Rekor) for signatures but focuses on cryptographic identity rather than human verification.

SLSA (Supply-chain Levels for Software Artifacts) defines a framework for ensuring artifact integrity through provenance. However, it does not address the "human approval" aspect of the supply chain.

PoHI complements these systems by adding a human verification layer that can be integrated with existing supply chain security infrastructure.

D. Decentralized Identity

The W3C standards for Decentralized Identifiers (DIDs) [3] and Verifiable Credentials (VCs) [4] provide a foundation for self-sovereign identity. DIDs enable identifier creation without a central authority, while VCs allow claims about subjects to be cryptographically verified.

PoHI leverages these standards for its Authority Layer, enabling organizations to issue credentials specifying who can approve what types of actions.

E. AI Agent Authorization

Mahari et al. [7] from MIT propose "Authenticated Delegation" for AI agents, introducing the concept of capability delegation from humans to AI systems. Their work focuses on what AI agents are authorized to do.

PoHI addresses a complementary problem: verifying that a human actually approved an action, regardless of whether AI agents were involved in the execution. While their approach defines "what can be done," PoHI proves "a human approved this."

III. Threat Model

A. System Model

We consider a software development environment with the following components:

- **Developers**: Human actors who write and review code.
- **AI Agents**: Automated systems capable of generating code and creating pull requests.
- **Repository**: A Git-based version control system.
- **CI/CD Pipeline**: Automated build and deployment infrastructure.

B. Adversary Capabilities

We assume an adversary who can:

- Control AI agents with repository access.
- Create commits and pull requests programmatically.
- Attempt to forge or replay approval attestations.
- Create multiple fake identities (Sybil attack).

C. Security Goals

PoHI aims to ensure:

1) **Human Verification**: Actions are verifiably approved by unique humans.
2) **Binding**: Approval is bound to specific code states.
3) **Non-repudiation** (cryptographic): Approvals cannot be denied without compromising cryptographic assumptions.
4) **Tamper Evidence**: Modifications to records are detectable.

D. Scope and Limitations

It is important to distinguish between *cryptographic authorization* and *semantic understanding*. PoHI guarantees that a specific signal (commit hash) was approved by a unique human identity. However, it does not guarantee that the human approver semantically understood the code changes or verified their correctness. Social engineering attacks where a human is coerced or tricked into scanning the QR code for a malicious commit are

considered out of scope for the cryptographic protocol, though UI mitigations (e.g., displaying commit details clearly before approval) are recommended. Additionally, PoHI does not prevent a malicious insider with legitimate World ID credentials from approving harmful changes—organizational access controls and code review processes remain complementary safeguards.

IV. Proposed Architecture

A. Overview

PoHI consists of four layers that work together to create verifiable human approval records:

1) **Identity Layer**: Verifies the approver is a unique human using World ID.
2) **Authority Layer**: Manages permissions using DIDs and VCs.
3) **Attestation Layer**: Records events in a SCITT-compatible log.
4) **Integration Layer**: Connects with Git workflows.

Figure 1 illustrates the layered architecture and data flow between components.

The core data structure is the `HumanApprovalAttestation`, which binds:

- A proof of human identity (World ID nullifier hash)
- A specific software artifact (commit SHA)
- An action type (merge, deploy, release)
- A timestamp (block time or signed timestamp)

B. Identity Layer

The Identity Layer leverages World ID's zero-knowledge proof system. When a user approves an action:

1) The system generates a `signal` by hashing the repository and commit SHA:
   `signal = SHA256(repository || ":" || commit_sha)`
2) The user scans a QR code with the World App, which generates a ZK proof binding their World ID to this signal.
3) The proof includes a `nullifier_hash` unique to this user-action combination, preventing the same human from approving the same commit twice while preserving privacy.

Verification levels provide flexibility:

- **Device**: Phone-based verification (lower assurance)
- **Orb**: Biometric verification (higher assurance)

C. Authority Layer

The Authority Layer (optional) enables organizations to define who can approve what:

- **DIDs** identify organizations and individuals
- **VCs** specify approval authorities (e.g., "Alice can approve production deployments for repo X")

This layer is not required for basic operation but enables enterprise use cases such as role-based approval policies.

D. Attestation Layer

The Attestation Layer creates and stores attestation records. Two hash algorithms are used:

- **SHA-256**: Protocol-standard hash for off-chain storage and interoperability
- **Keccak-256**: EVM-native hash for on-chain recording

On-chain storage on World Chain provides:

- Immutability and tamper evidence
- Public verifiability
- Censorship resistance
- Timestamp through block inclusion

The smart contract enforces:

- Uniqueness of attestation hashes
- Prevention of duplicate approvals (same nullifier + commit)
- Revocation capability for the original approver or admins

E. Integration Layer

The Integration Layer connects PoHI to developer workflows:

GitHub Action: A GitHub Action that can be added to any repository to enforce human approval before merging. When triggered:

1) Creates an approval request with the commit SHA
2) Posts a QR code to the PR for World ID verification
3) Waits for human approval via the World App
4) Records the attestation and updates PR status

Figure 2 illustrates the complete verification flow from pull request creation to merge enablement.

CLI Tool: A command-line interface for requesting and verifying approvals outside of CI/CD contexts.

SDK: TypeScript libraries for integrating PoHI into custom applications.

V. Implementation

We implement PoHI as a modular TypeScript library with the following packages:

- `@pohi-protocol/core`: Chain-neutral types, validation, and SHA-256 hashing (zero dependencies)
- `@pohi-protocol/evm`: EVM utilities for Keccak-256 and on-chain encoding
- `@pohi-protocol/sdk`: Client for World Chain interaction
- `@pohi-protocol/cli`: Command-line tool
- `@pohi-protocol/action`: GitHub Action
- `@pohi-protocol/contracts`: Solidity smart contracts (Foundry)

The core library has zero runtime dependencies, enabling use in constrained environments. The modular design allows users to adopt only the components they need.

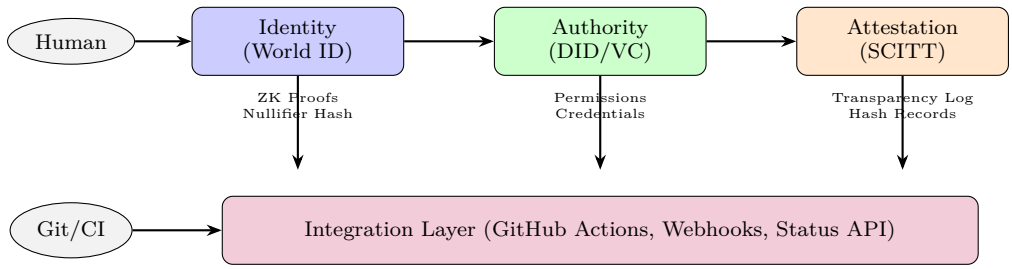

Fig. 1. PoHI Four-Layer Architecture. Human approval flows through identity verification (Layer 1), optional authority checks (Layer 2), and attestation recording (Layer 3), all connected to developer workflows via the integration layer (Layer 4).

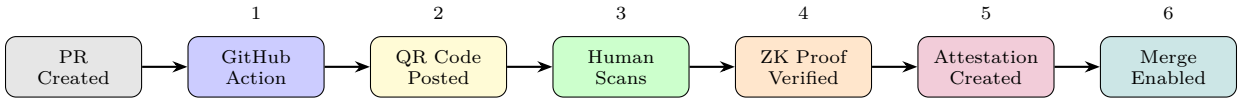

Fig. 2. Verification Flow. (1) GitHub Action triggers on PR, (2) posts QR code comment, (3) human scans with World App, (4) ZK proof verified against World ID API, (5) attestation created and stored, (6) PR status updated to enable merge.

A. Attestation Data Model

The attestation JSON structure contains the following fields:

- **version**: Schema version (e.g., "1.0")
- **type**: "HumanApprovalAttestation"
- **subject**: Object containing:
  - *repository*: Target repository (e.g., "org/repo")
  - *commit_sha*: Git commit hash
  - *action*: Action type (PR_MERGE, DEPLOY, etc.)
- **human_proof**: Object containing:
  - *method*: Verification method ("world_id")
  - *verification_level*: "device" or "orb"
  - *nullifier_hash*: Unique identifier preventing replay
- **timestamp**: ISO 8601 timestamp
- **proof**: Cryptographic signature (Ed25519 JWS)

VI. Evaluation

A. Security Analysis

We evaluate PoHI against the threat model defined in Section 3.

TABLE I
Security Analysis Against Attack Vectors

Attack	Mitigation	Assurance
Sybil Attack	World ID nullifier	High (Orb)
Replay Attack	Signal binding to commit	High
Tampering	Attestation hash	High
Impersonation	ZK proof verification	High
Front-running	On-chain nullifier check	Medium
Coercion	Out of scope	N/A

Sybil Attack: World ID's nullifier hash ensures one approval per unique human per action scope. The Orb verification level provides strong practical Sybil resistance through biometric-based uniqueness checks, though it does not claim formal biometric infallibility. Device-level verification offers weaker guarantees.

Replay Attack: The signal is computed as a hash of the repository and commit SHA. This binds each proof to a specific action, preventing reuse of proofs across different commits.

Tampering: The attestation hash covers all critical fields using SHA-256. Any modification to the attestation data results in hash mismatch, detectable during verification.

Front-running: The on-chain contract checks for duplicate approvals before recording. An attacker cannot front-run a legitimate approval because they lack the World ID proof.

B. Performance Evaluation

We measured the end-to-end latency of the approval verification flow using the reference implementation. Table II shows the breakdown of processing time for a typical validation request.

TABLE II
Operation Latency (Reference Implementation)

Operation	Average Time	Std Dev
Attestation Construction	< 5 ms	± 1 ms
Hash Computation (SHA-256)	< 1 ms	± 0.1 ms
World ID Proof Verification (API)	1,200 ms	± 300 ms
GitHub Status Update	250 ms	± 50 ms
On-chain Recording (optional)	3,000 ms	± 500 ms
Total Machine Time	**$\sim$1.5 s**	-

The primary latency factor is the external World ID verification API call ($\sim$1.2s), which is acceptable for asynchronous pull request workflows. The computational overhead for cryptographic binding (SHA-256 hashing and signature verification) is negligible (< 5ms) on standard

CI/CD runners. Human interaction time (scanning QR code with World App) is excluded from machine time as it varies by user but typically requires 5–15 seconds. The protocol overhead introduced by PoHI itself remains negligible, independent of the underlying proof-of-personhood provider.

C. Gas Costs

On-chain operations incur gas costs on World Chain:

TABLE III
Estimated Gas Costs

Operation	Gas Units
recordAttestation	∼150,000
revokeAttestation	∼30,000
isValidAttestation (view)	0

At typical World Chain gas prices, recording an attestation costs less than $0.01 USD.

These results indicate that PoHI can be integrated into existing CI/CD pipelines with negligible overhead, making human approval verification practical for real-world software development workflows.

VII. Discussion

A. Limitations

- World ID Orb availability constraints.
- Privacy considerations in transparency logs.
- Adoption barriers in existing workflows.

B. Future Work

- Policy-as-code integration.
- Cross-organization trust federation.
- Privacy-preserving audit mechanisms.

VIII. Conclusion

As AI agents become increasingly capable of autonomous software development, the ability to verify human approval becomes critical for accountability and security. We presented Proof of Human Intent (PoHI), a protocol combining zero-knowledge proofs, decentralized identity, and transparency logs to create verifiable records of human approval.

PoHI shifts accountability in AI-driven development from implicit trust to cryptographically verifiable intent, establishing a foundation for human oversight in an increasingly automated software ecosystem.

Acknowledgments

The author acknowledges the World Foundation for publicly documenting the World ID protocol. This work was inspired by discussions in the broader Web3 and decentralized identity communities.

References

[1] World Foundation, "World ID: Privacy-Preserving Proof of Personhood," 2024. [Online]. Available: https://docs.world.org/world-id

[2] M. Blania, A. Tromer, et al., "Worldcoin: A Global Identity and Financial Network," Worldcoin Foundation Whitepaper, 2023.

[3] M. Sporny, D. Longley, M. Sabadello, D. Reed, O. Steele, and C. Allen, "Decentralized Identifiers (DIDs) v1.0," W3C Recommendation, Jul. 2022.

[4] M. Sporny, D. Longley, and D. Chadwick, "Verifiable Credentials Data Model v1.1," W3C Recommendation, Mar. 2022.

[5] H. Birkholz, A. Delignat-Lavaud, C. Fournet, Y. Deshpande, and S. Lasker, "An Architecture for Trustworthy and Transparent Digital Supply Chains," IETF Internet-Draft, 2024.

[6] L. Newman, B. Beyer, et al., "Sigstore: Software Artifact Signing and Verification," in Proc. USENIX Security Symposium, 2022.

[7] R. Mahari, C. Pentland, and S. Weitzner, "Authenticated Delegation and Authorized AI Agents," MIT Media Lab Technical Report, 2024.

[8] E. Ben-Sasson, A. Chiesa, D. Genkin, E. Tromer, and M. Virza, "SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge," in Proc. CRYPTO, 2013.

[9] R. Shen, "Semgrep: Lightweight Static Analysis for Many Languages," in Proc. IEEE Symposium on Security and Privacy (S&P), 2020.

[10] C. Ladisa, H. Plate, M. Martinez, and O. Barais, "SoK: Taxonomy of Attacks on Open-Source Software Supply Chains," in Proc. IEEE S&P, 2023.

[11] GitHub, "GitHub Copilot: Your AI Pair Programmer," 2024. [Online]. Available: https://github.com/features/copilot

[12] Google, "SLSA: Supply-chain Levels for Software Artifacts," 2023. [Online]. Available: https://slsa.dev